

TEMA 3_1 – LINUX 进程和任务

进程 - 任务

1. 进程	2
1.1. 进程的概念	2
1.2. 进程创建	3
1.3. 进程执行	3
1.4. 进程终止	4
1.5. 进程类型	5
1.6. 进程状态	6
1.7. 进程的实现	6
1.8. 线程的概念	7
2. LINUX. 进程管理	8
2.1. 管理进程的命令	11
2.2. 图形环境下管理进程	21
2.3. 虚拟目录 /proc	22
2.4. 特殊权限文件	23
2.5. 其他命令 - 系统性能	24
3. LINUX. 引导过程 – 服务管理	26
3.1. 引导过程 - 阶段	26
3.2. 获取引导过程信息	28
3.3. System V 初始化过程	29
3.4. Systemd 初始化过程	32
4. LINUX. 任务自动化	36
4.1. 在特定时间执行任务：at	36
4.2. 定期执行任务：cron	38
4.3. Anacron	40

1. 进程

1.1. 进程的概念

一个进程本质上是一个正在执行的程序。

一个进程与一个程序并不相同。一个程序是一组指令和数据，存储在一个文件（可执行文件）中。当这个程序的内容被加载到内存中并开始执行时，它就变成了一个进程。

一个进程是一个由操作系统创建的、动态活跃的实体，用于执行一个程序。每个进程都关联着一个地址空间（分配给进程的主内存），其中包含可执行程序、程序数据和执行栈。

每个进程还关联着一组资源，通常包括 CPU 寄存器（程序计数器等）、打开的文件列表、待处理的闹钟、相关进程列表以及其他所有执行程序所需的必要信息。本质上，一个进程是一个容器，用于存储执行程序所需的所有信息。

同一个程序运行两次会产生两个不同的进程（尽管它们共享存储程序代码的主内存区）。

操作系统管理进程，并提供并发（伪）操作的能力，即使在只有一台 CPU 可用的情况下也是如此。

现代操作系统是多任务或多程序的，并允许有多个或许多活跃进程（视频编辑器、网络浏览器、杀毒软件等）。

CPU 快速地在进程之间切换，给人一种伪并行的感觉（因为在严格意义上，任何时刻一台 CPU 只执行一个进程*）。

进程调度器通过调度算法将决定何时停止一个进程以执行另一个进程。当操作系统决定暂时停止一个进程的执行时，必须保存该进程的所有信息，因为之后必须以停止时的状态精确地重新启动。

* 注：当前限制是，在任何时刻，每个处理器核心只能存在一个正在执行的进程。多处理器、多核或支持多线程的系统允许并行执行多个进程。然而，它们使用相同的

时间片轮转系统来管理存在更多活跃进程而非可用处理器核心的情况。

因此，在多任务操作系统中，有多个程序加载到主内存中，因为所有进程要执行都必须在内存中。它们独立地运行，也就是说，无论是否有其他程序同时运行，一个程序都以相同的方式执行。然而，当一个任务被划分为多个相互协作以完成该任务的进程时，它们需要通信，以传递数据和指令，并在执行其动作时同步。

一个进程的生命周期可以分为以下几个阶段：

- 进程被创建。
- 进程被执行。
- 进程结束或死亡。

1.2. 进程创建

导致进程创建的主要事件有：

- 系统启动：通常，当操作系统启动时，会创建多个进程。其中一些是前台进程，与用户交互（例如：终端、图形界面等）。另一些是后台进程，与特定功能相关联；它们在后台运行，大部分时间处于等待状态（例如：打印、接收邮件等）。
- 从进程执行一个系统调用以创建新进程。
- 用户创建进程的请求：用户可以通过输入命令或（双）击图标来启动程序。
- 批处理（batch）进程的启动。

技术上，在所有这些情况下，要创建进程，需要让现有进程执行创建进程的系统调用。该系统调用指示操作系统创建进程，并直接或间接地告知其应执行哪个程序。

在 Unix/Linux 中，只有一个系统调用用于创建进程：fork。

- fork()：一个进程（父进程）复制其上下文并创建一个几乎相同的子进程。
- exec()：子进程（或同一个进程）用一个新的程序（二进制/ELF 或脚本）替换其映像。

- `clone()`: 低级变体，用于创建具有共享资源空间的线程或进程（Linux 线程的基础）。

这个调用会创建一个与调用进程完全相同的进程副本。

无论是在 Unix/Linux 还是 Windows 系统中，一旦创建了一个进程，父进程和子进程都会有各自独立的地址空间。如果任何一个进程修改了其地址空间中的某个字，这种修改对另一个进程是不可见的。

在某些系统中，当一个进程创建另一个进程时，父进程和子进程会继续保持关联。子进程可以创建更多的进程，从而形成一个进程层次结构：

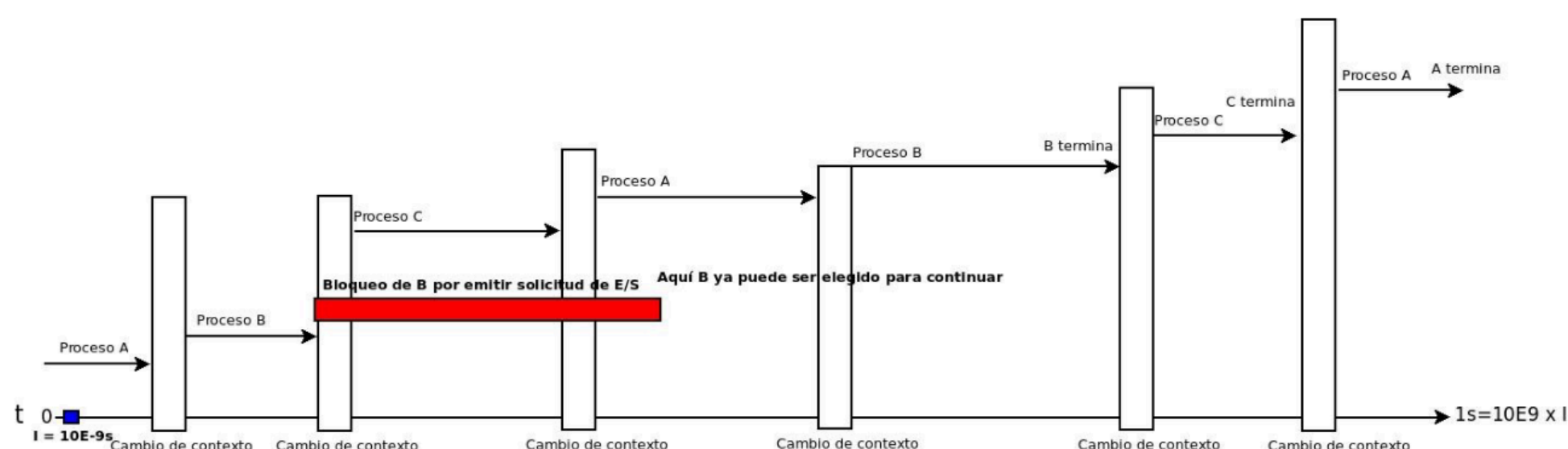
- 在 Unix/Linux 中，一个进程及其所有子进程，连同其后续的子孙进程，会形成一个进程组。Unix/Linux 中的进程不能剥夺其子进程的继承权。

1.3. 进程执行

进程的目标正是执行。这种执行包括处理突发和等待突发。

当进程正在执行时可能会发生中断。无论是外部中断、异常还是系统调用（进程请求操作系统服务），进程都会停止执行。立即进入操作系统执行（用户模式切换到特权模式）。稍后操作系统将激活此进程，即继续其之前的状态（切换回用户模式）。

操作系统不断将一个进程从 CPU 中移出并分配给另一个进程。这种操作称为上下文切换。上下文切换需要保存被移出进程的状态，以便稍后可以像什么都没发生一样继续执行。当 CPU 分配给一个进程时，会恢复其所有状态。



CPU 在“分配”给准备执行的进程之间：A、B、C

操作系统决定哪个进程应该运行以及运行多长时间的部分是进程调度器。进程调度器负责处理中断以及启动和停止进程的相关细节。进程调度非常重要；人们设计了多种算法来确保操作系统尽可能高效。所使用的算法被称为进程调度算法。其中一些算法使用进程的优先级，这是一个数值，用作确定一个进程是否应该在另一个进程之前进入 CPU 的依据。进程之间经常需要相互通信，以传输数据和指令，以同步其活动。这种通信被称为进程间通信（IPC）。可能会发生以下情况：

- 一个进程需要将信息传递给另一个进程。
- 需要控制两个或多个进程之间不相互干扰。例如，在一个航班预订系统中，两个进程都在尝试为不同的客户获取最后一个可用座位。
- 需要保证进程之间的适当顺序，当它们之间存在依赖关系时。例如，如果进程 A 生成数据，而进程 B 打印这些数据；B 必须等待 A 生成一些数据后才能开始打印。

IPC 提供了一种机制，允许进程之间进行通信和同步：共享内存、消息传递、信号量等..

1.4. 进程终止

一旦创建了一个进程，它就开始执行并完成其分配的任务。迟早，新的进程会终止，通常是由于以下条件之一：

- 正常输出（自愿性）。大多数进程由于完成了工作而终止。此时它们执行一个系统调用，通知操作系统已结束。这个调用在 Unix/Linux 中称为 `exit`，在 Windows 中称为 `ExitProcess`。
- 错误输出（自愿性）。进程发现错误，例如请求一个不存在的文件时，就会终止。
- 致命错误（非自愿性）。通常由程序错误引起，例如除以零。
- 被其他进程删除（非自愿性）。在 Unix/Linux 中，此调用为 `kill`。Win32 中相应的函数是 `TerminateProcess`。在这两种情况下，删除进程的进程必须具有执行删除操作的权限。

1.5. 进程类型

通常由于以下一种或多种条件：我们可以根据不同的标准对进程进行分类：

- 根据其设计：
 - 可重用型：每次使用时都会加载到内存中。用户程序通常属于此类。
 - 可重入型：将代码的一个副本加载到内存中。每次使用时都会创建一个新进程，拥有自己的数据区，但共享代码。例如，并发执行多个 `ls` 操作。
- 根据其对 CPU 和资源的访问方式：
 - 抢占式：可以访问资源，并且只有在自愿（通过 CPU 指令）的情况下才放弃这些资源。
 - 非抢占式：允许其他进程抢占其当前拥有的资源
- 根据其在内存中的持续时间：
 - 居民：需要在整个生命周期中（从创建到终止）保持在内存中。
 - 可交换的（swappable）：这是最常见的情况。操作系统可以在其生命周期中决定将它们移至磁盘。
- 根据其所有者：
 - 用户进程：由用户设计。在非保护/非特权模式下运行。
 - 系统进程：是指构成操作系统的一部分的进程（如输入输出、其他进程的计划等）。

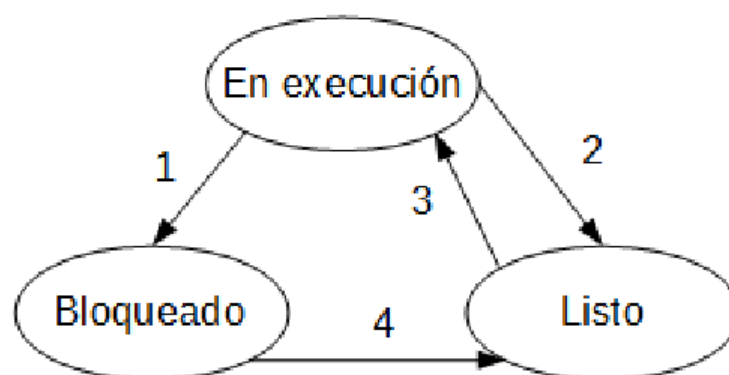
- 根据执行方式：
 - 前台进程：是与用户交互的进程，即通过终端接收用户信息，并通过同一终端返回结果。只能有一个进程在前台执行。
 - 后台进程：是不与用户交互的进程。可以有多个进程在后台执行。

1.6. 进程状态

一个进程在其生命周期中可能处于不同的状态。尽管不同操作系统中的状态数量和名称可能有所不同，但它们都遵循一个通用模式。

一个进程可以处于以下状态：

- 运行中。该进程正在使用处理器。
- 就绪或准备。可执行；该进程暂时停止以允许另一个进程执行。
- 阻塞。该进程在发生特定外部事件之前无法执行。此状态也称为睡眠或等待。



该图展示了进程可能存在的三种状态以及这些状态之间的转换（1、2、3、4）：

1. 当进程请求输入/输出操作或其他事件时，它会被阻塞，并停止使用处理器。
2. 调度器会选择另一个进程。
3. 调度器会选择该进程并开始使用处理器。
4. I/O 操作或其他事件，进程一直在等待的，已经结束了。

- 过渡 2 发生在调度器决定正在执行的过程已经运行了足够长的时间，是时候让另一个进程使用 CPU 部分时间的时候。
- 转换 3 发生在其他所有进程都获得了它们的部分 CPU 时间之后，此时第一个进程获得 CPU 以重新执行。

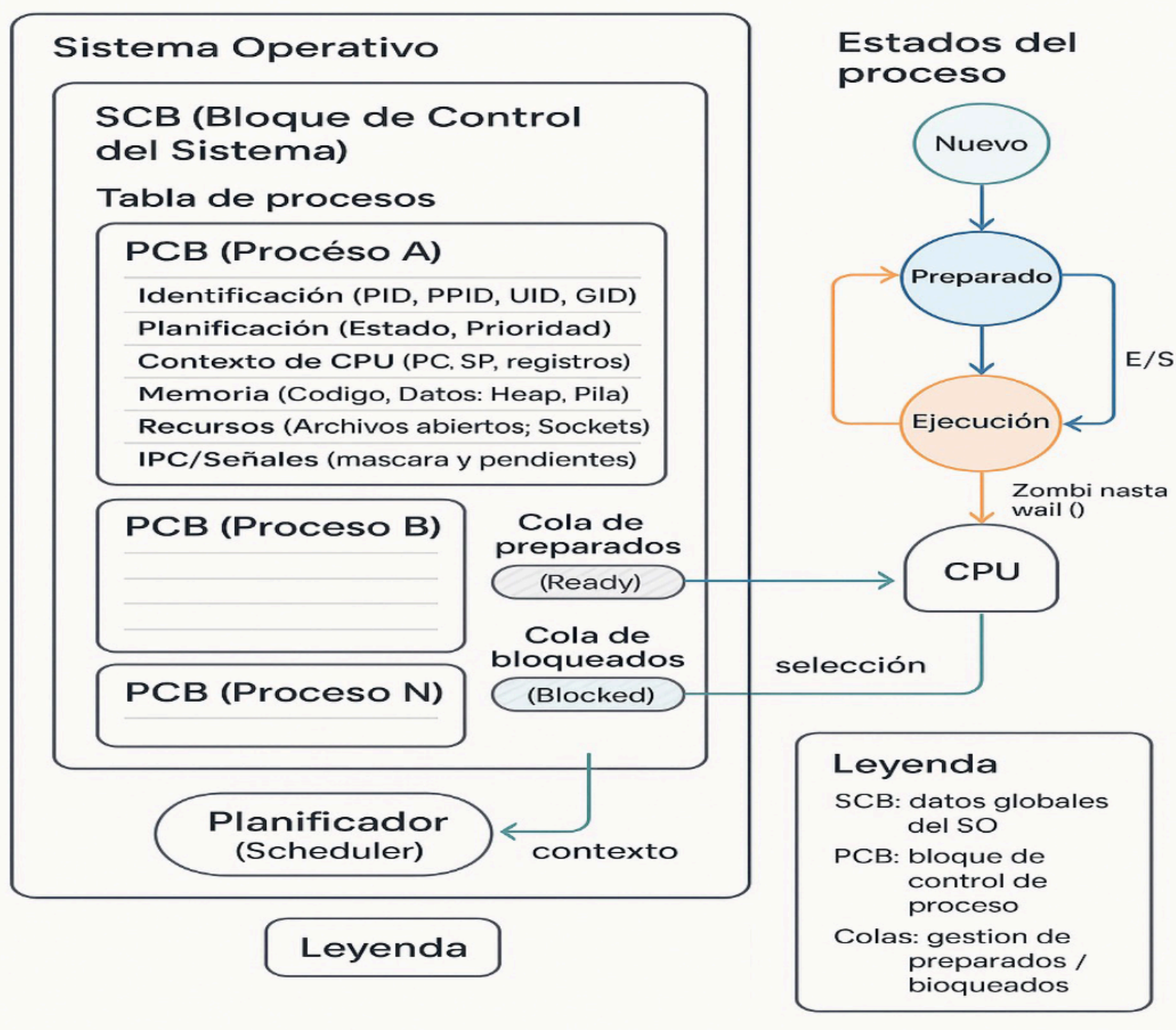
1.7. 进程的实现

为了实现进程模型，操作系统需要知道它拥有的进程列表以及它们的状态。

这些数据存储在系统控制块（SCB）中：操作系统用于管理进程执行的一组数据结构。

此外，操作系统管理一个称为进程表（process table）的表格，用于存储每个进程的相关信息。该表格的每一项被称为进程控制块（PCB），其中包含操作系统用于管理该进程的每个进程的特定数据。

Modelo de procesos (SO)



PCB 通常包括以下信息：

- 标识 (ID)：进程 ID、父进程 ID、用户和组 ID
- 调度：进程状态、进程优先级……
- 分配的内存：数据段、代码段、栈的指针。
- 分配的资源：打开的文件……
- 用于进程间通信的信息：信号，……

1.8. 线程概念

我们可以将线程、线程或子线程定义为可由操作系统调度的最小处理单元。它是一系列指令。

在传统操作系统中，每个进程都有一个地址空间和一个线程（单线程）。现代操作系统

当前操作系统支持在同一个地址空间中运行多个线程的进程，这些线程以准并行（多线程）方式执行。

多线程进程的优势如下：

- 许多应用程序同时开发多种活动。
将应用程序分解为多个顺序执行的线程，这些线程近似并行运行，编程模型得以简化。
- 由于线程比进程更轻量，因此创建（即快速）和销毁线程更加容易。
- 当应用程序有大量计算和 I/O 操作时，执行速度会更快；通过使用线程，这些活动可以重叠进行。
- 在多 CPU 系统中，可以实现真正的并行（每个处理器将执行一个线程）。

一个展示这些优势的例子是考虑一个正在使用文字处理器的用户：一个线程与正在输入文本的用户交互，另一个线程定期将 RAM 中的内容保存到磁盘（自动保存），第三个线程可能正在对文档的其他部分应用文本格式更改。这三个线程都修改文档；也就是说它们共享一个公共内存（如果它们是分离的进程将无法工作）。

线程为进程模型带来的好处是允许在同一进程环境中执行多次运行，这些运行在很大程度上是相互独立的。线程和进程之间存在相似之处：

- 线程共享一个地址空间和其他资源；
- 进程共享物理内存、磁盘和其他资源。
- 线程在一个进程内部伪并发地执行；
- 进程在计算机上伪并发地执行。
- 多线程的工作方式与多任务（多道程序设计）相同：CPU 在不同线程之间进行切换。

线程与进程的区别在于它们不是那么像进程这样的独立实体。通过共享更多资源（地址空间、所有者用户、打开的文件等），在同一进程内更容易在多个线程之间切换（因为需要保护的信息更少）。线程之间的通信比进程之间的通信更为常见。

如同一个传统进程（即只有一个线程的进程），一个线程可以处于以下几种状态之一：运行中、阻塞、就绪或终止。

2. LINUX. 进程管理

- 在 Linux 中，进程结构是分层的。为了表示进程之间的关系，使用父进程-子进程的概念。例如，当从 shell 中执行一个命令时，我们正在启动另一个进程，该进程将是父进程（shell）的子进程。
- 在 Linux 系统中运行的每个进程都会被分配一个唯一的标识符，称为进程 ID（PID）。PID 是一个整数。除了 PID 之外，进程还会被分配另一个编号，称为父进程 ID（PPID），用于标识该进程的父进程。
- 每个进程都与一个用户相关联，包括 UID（用户标识号），以及一个或多个组，包括 GID（组标识号），这些决定了进程访问系统资源和文件的权利。
- 在 Linux 中，可以在两个进程之间创建通信通道；一个进程可以写入字节流供另一个进程读取。这些通道被称为管道（pipelines）。Shell 命令行中的管道通过管道符号|实现。进程也可以以其他方式通信：一个进程可以向另一个进程发送所谓的信号。
- 进程调度器使用优先级。Linux 内核使用动态算法来计算优先级。它也关注一个由管理员设定的值，通常被称为“nice 值”。
- 在 Linux 中，进程的不同状态按以下方式识别：
 - 正在运行 / Running (R)。该进程正在执行。
 - 锁定 / 休眠 / 休眠 (S)。- 进程处于等待某种类型的事件（例如，接收软件或硬件信号）。一旦该事件发生，进程将开始执行/继续。
 - 停止 / 跟踪 (T)。- 一个停止状态的过程不是在等待任何事件。它的执行已经被停止，只有当收到一个允许其继续执行的特定信号时，才会继续执行。
 - Zombie (Z)。- 每个进程在结束时都会通知其父进程，以便父进程从进程表中删除其条目。如果

父亲因为某种原因没有接收到这条通信，因此不会从进程表中删除它。在这种情况下，子进程会处于僵尸状态，或者称为已终止状态，它不会消耗 CPU，但仍然会继续消耗系统资源。

在 ° (D) 的情况下。进程处于不可中断的睡眠/停滞状态。
(不可中断的睡眠)。这种状态通常是暂时的。

- 用户执行他们的程序：在图形环境中双击；在命令解释器中输入可执行文件的名称及其选项和参数。在这种情况下，需要注意 Linux 不将当前目录包含在 PATH 变量中，因此为了执行当前目录中的程序（该程序未包含在 PATH 变量中），我们将输入 `./nombre_del_programa`。
- 每个用户始终有一个活跃的进程：即 shell 本身。如果我们执行一个命令，我们将启动另一个进程，该进程将是启动该命令的进程的子进程。
- 在 Linux 中，由用户从 shell 中执行的进程被称为作业或任务 (jobs)。因此，一个任务仅仅是 shell 的工作单元。
- 一个用户可以同时运行多个作业。然而，其中只有一个能够直接与用户交互。这个作业被称为前台作业（或前景作业）。其余的作业可以在被称为后台（或后台）的地方运行。尽管用户不能直接与后台进程交互，但如果其中一个作业生成输出，该输出会出现在终端（为避免这种情况，需要重定向该输出）。如果后台作业在运行时需要输入，该作业将停止或暂停。
- 前台进程 (foreground)：当 shell 处于程序/命令执行完成后的休眠状态，因此终端在子进程结束前不接受新的程序/命令。要在前台启动一个进程，可以正常执行。

示例：`ls -l /var/log > ls_log`

° 在前台执行一个进程后，在它执行完毕并返回提示符之前，我们将无法输入新的命令。

- ° 前台运行的程序可以通过按下 Ctrl-C 组合键来终止。这样我们就可以结束正在运行的进程。
- ° 我们也可以通过按下 Ctrl-Z 暂停/停止前台正在执行的过程。这样过程不会终止，而是进入永久阻塞的后台状态。（屏幕显示分配的工作号，状态为停止）

```
$mousepad
^Z
[1]+  Detenido          mousepad
$
```

暂停/停止/终止过程的示例

- ° 不需要长时间执行且需要与用户交互的程序通常在前台启动。
- 后台进程（background）：如果子进程或多个子进程与父级 shell 进程并发执行，父级 shell 进程将继续接受新的程序/命令。
在后台启动一个进程，需要在命令末尾添加字符&。

示例：ls -l /var/log > ls_log &

- ° 在后台执行时，shell 会打印一行，例如
[1] 6190 并返回到提示符：
 - ° 方括号内的数字是 shell 将进程置于后台时分配的工作或任务编号（一个整数，从 1 开始按顺序编号）
 - ° 另一个数字是分配给进程的进程号，即 PID。
- ```
$mousepad &
[1] 2237
$
```
- ° 与后台进程通信的唯一方式是通过发送信号。
  - ° 当我们从终端运行图形程序（gedit 等）时，通常将其置于后台运行，以便我们可以执行其他程序/命令。通常也会将那些运行时间较长且不需要与用户交互的程序置于后台执行。

- ° 它们也被称为批处理模式下的进程。
- 除了由用户执行的进程（前台=交互式或后台=批处理进程）之外，在 Linux 系统中还可以运行其他类型的进程：



在一个 Linux 系统中，可以运行其他类型的进程：° 守护进程（Daemons）：在后台持续运行的进程（也就是说，直到计算机关闭或会话结束之前永远不会终止）。许多守护进程在系统启动时被启动，然后等待用户或系统的请求，表明需要其服务。

恶魔通常具有以下特征：

- 它们既没有图形界面也没有文本界面供用户使用
- 它们既不与用户进行输入交互也不进行输出交互。
- 它们的运行记录和错误通信都在通常位于 /var /log 的日志文件中完成。

一个托管着 Web 服务器的计算机将使用一个守护进程来提供该服务。另一个例子：允许在特定时间条件下（例如每周五）执行任务（如系统维护）的“计划任务”守护进程。

° 内核中的进程，用户既不能开始也不能结束，并且对它们几乎没有控制权。

° 有其他用户在其他终端上启动的进程。

## 2.1. 管理进程的命令

```
ps
```

- 命令 ps（进程状态），用于显示当前系统中正在运行的进程信息。
- 语法：ps [选项]
- 没有选项时，仅显示当前用户在相应 shell（即当前终端）中运行的进程。
- ps 语法的简洁性隐藏了相当大的复杂性，因为 ps 支持三种不同类型的选项，以及每种类型中包含的许多选项。ps 支持的三种选项类型是：
  - ° System V 风格 选项前缀为短横线→° BSD 风格 选项不带短横线。
  - ° GNU 风格 使用长选项名，选项前缀为双横线-

- System V 版本。ps 命令最常用的选项是：-e 显示系统中所有进程的信息。

- 这个选项与选项 -e -l 长格式列表（许多进程的详细信息） -f 完整格式列表（许多进程的详细信息） -u 用户 显示属于特定用户的进程 示例：ps -ef

- BSD 版本。最常见的选项有：

a 也显示其他用户的进程 x 也显示没有关联终端的进程 l BSD 长格式列表 u 用户  
格式列表 m 显示与内存相关的信息 U 用户 显示属于特定用户的进程

示例：ps ax

Otro Ejemplo: ps -f -u tty02

- GNU 版本：最常见的选项有：

--help 显示常用 ps 选项的摘要 --forest 显示进程间关系的层次结构 --user 用户  
显示属于特定用户的进程

- 可以混合使用 SystemV 和 BSD 的选项，但存在一些冲突。也有一些同义词选项（功能相同）

示例：查看所有进程

| 语法<br>systemV              | 语法<br>BSD       |
|----------------------------|-----------------|
| ps -e<br>ps -ef<br>ps -efl | ps ax<br>ps aux |

```
$ ps -el
F S UID PID PPID C PRI NI ADDR SZ WCHAN TTY TIME CMD
4 S 0 1 0 0 80 0 - 13236 - ? 00:00:01 systemd
1 S 0 2 0 0 80 0 - 0 - ? 00:00:00 kthreadd
1 S 0 3 2 0 80 0 - 0 - ? 00:00:00 ksoftirqd/0
1 S 0 5 2 0 60 -20 - 0 - ? 00:00:00 kworker/0:0H
1 S 0 7 2 0 80 0 - 0 - ? 00:00:00 rcu_sched
1 S 0 8 2 0 80 0 - 0 - ? 00:00:00 rcu_bh
```

- 命令 ps 加上选项 l 可以显示相关的属性对进程。每个字段的含义如下：
  - S 每个进程关联的状态：R、T、S、Z、D。（已解释）
  - UID 用户拥有的进程的标识号
  - PID 进程的标识号
  - PPID 父进程的识别号
  - C 表示进程使用处理器的程度。是一个整数，表示处理器使用百分比 (%CPU)。
  - PRI 与进程关联的优先级
  - NI 使用的 nice 值。默认值为 0。正值表示优先级降低，负值表示优先级提高。
  - ADDR 进程内存地址
  - SZ 进程块大小
  - WCHAN 进程等待事件。如果为空，表示进程正在运行。
  - TTY 关联的终端。某些进程不关联任何终端，此时该列显示问号？。
  - TIME 总计消耗的处理器时间。
  - CMD 执行并导致进程产生的命令（其他列表中的 COMMAND）
- 执行 ps 命令时出现的其他字段的意义：
  - USER 进程所有者的用户名。
  - %CPU 程序使用的处理器利用率百分比。
  - %MEM 程序使用的内存利用率百分比。
  - STIME / START 程序开始的时间。
  - VSZ / VIRT 程序在虚拟内存中的图像大小（以 KiB 为单位）。

- RSS / RSZ 指示程序在内存中驻留的千字节数量
- STAT 进程状态
- 由于在 Linux 系统中同时运行着大量进程，因此能够根据某些标准查找进程非常实用，例如由某个用户启动的进程等。为此，我们将使用 grep 命令：将 ps 命令的输出与 grep 连接起来，以获取仅包含特定文本的进程。

top

- ps 命令显示的是我们在执行命令时的进程“快照”。top 命令是 ps 命令的交互式版本。
- top 命令实时显示系统中正在运行的过程列表。它还显示系统状态的摘要。
- 该命令是了解某些进程相对于其他进程消耗多少 CPU 时间，或查看哪些进程消耗更多 CPU 时间的良好工具。默认情况下，top 按 CPU 使用率对其输入进行排序，每隔几秒钟更新一次显示的数据。
- 执行不带选项的 top 时，屏幕分为三个区域：系统信息区域、消息/提示符行和进程列表。

```
top - 23:08:23 up 7 min, 4 users, load average: 0,03, 0,16, 0,12
Tasks: 145 total, 1 running, 144 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0,0 us, 0,3 sy, 0,0 ni, 99,7 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
KiB Mem: 1020372 total, 552724 used, 467648 free, 40012 buffers
KiB Swap: 1768444 total, 0 used, 1768444 free. 264144 cached Mem
```

| PID  | USER     | PR | NI  | VIRT   | RES   | SHR   | S | %CPU | %MEM | TIME+   | COMMAND      |
|------|----------|----|-----|--------|-------|-------|---|------|------|---------|--------------|
| 1464 | root     | 20 | 0   | 247108 | 52396 | 27208 | S | 0,3  | 5,1  | 0:01.47 | Xorg         |
| 1878 | root     | 20 | 0   | 316564 | 20196 | 15052 | S | 0,3  | 2,0  | 0:00.29 | ica          |
| 1946 | adminis+ | 20 | 0   | 437036 | 24780 | 20720 | S | 0,3  | 2,4  | 0:00.33 | xfdesktop    |
| 2112 | adminis+ | 20 | 0   | 25724  | 2912  | 2420  | R | 0,3  | 0,3  | 0:00.64 | top          |
| 2274 | root     | 20 | 0   | 0      | 0     | 0     | S | 0,3  | 0,0  | 0:00.04 | kworker/0:0  |
| 1    | root     | 20 | 0   | 52936  | 6728  | 4816  | S | 0,0  | 0,7  | 0:01.05 | systemd      |
| 2    | root     | 20 | 0   | 0      | 0     | 0     | S | 0,0  | 0,0  | 0:00.00 | kthreadd     |
| 3    | root     | 20 | 0   | 0      | 0     | 0     | S | 0,0  | 0,0  | 0:00.05 | ksoftirqd/0  |
| 5    | root     | 0  | -20 | 0      | 0     | 0     | S | 0,0  | 0,0  | 0:00.00 | kworker/0:0H |
| 6    | root     | 20 | 0   | 0      | 0     | 0     | S | 0,0  | 0,0  | 0:00.00 | kworker/u2:0 |
| 7    | root     | 20 | 0   | 0      | 0     | 0     | S | 0,0  | 0,0  | 0:00.12 | rcu_sched    |
| 8    | root     | 20 | 0   | 0      | 0     | 0     | S | 0,0  | 0,0  | 0:00.00 | rcu_bh       |

Área de información do sistema

Liña de mensaxes / prompt

Lista de procesos

- 系统信息区域：
  - 第一行显示系统活动时间和平均负载。显示：当前时间、系统已运行时间、当前连接用户数和平均负载，分别为 1.5 分钟和 15 分钟的间隔（值 1 表示系统繁忙，但未过载）。



° 第二行显示了总进程数以及每个状态下的进程数：运行、睡眠、停止和僵尸状态。° 第三行以百分比显示了 CPU 的状态。

指示 CPU 时间：用户进程 (us)、内核进程 (sy)、低优先级用户进程 (ni)、不活动进程 (id)、等待输入/输出 (wa)、硬件中断 (hi)、软件中断 (si)、虚拟机管理程序时间 (st)。

° 第四行显示物理内存 (RAM) 的使用情况，分为：总内存、已用、空闲和缓冲区使用。° 第五行显示虚拟内存/交换区使用情况，分为：总内存、已用、空闲和缓存。

- 进程列表：

° 执行不带选项的 top 命令时，显示系统中正在运行的所有进程列表。它显示的信息列与 ps 命令相似，不同之处在于它会定期更新。

默认显示的列有：

- USER (USUARIO)：进程的所有者用户。
- PR：进程的优先级。如果设置为 RT，则表示正在实时执行。
- NI：分配优先级。如果值较低（直到-20），则表示比值为高（直到 19）的进程具有更高的优先级。
- VIRT：进程使用的虚拟内存量。
- RES：进程使用的物理 RAM 内存量。
- SHR：共享内存。
- S（状态）：进程的状态。
- %CPU: 自上次更新以来使用的 CPU 百分比。
- %MEM: 自上次更新以来进程使用的物理内存百分比。
- TIME+ (HORA+): 进程自启动以来使用的总 CPU 时间。
- COMMAND: 用于启动进程的命令。

° 通过消息行/提示符，我们可以隐藏或显示信息列，并选择要查看的进程。

- 消息行/提示符：° 从这行我们可以输入 top 交互式命令。这些命令是单字符的，尽管有些需要更多信息。它们可以执行多种操作。

最常用的有：

- h o ? : 显示帮助信息界面。  
我们需要输入 q 来退出此界面。
- q : 退出 top。
- k : 向一个进程发送信号 (kill 命令)。请求进程的 PID (默认为列表中的第一个) 和要发送的信号 (默认 15/SIGTERM)。
- r : 更改进程的优先级 (renice 命令)。请求进程的 PID (默认为列表中的第一个进程) 和 nice 值 (默认 15/SIGTERM)。
- f 或 F : 显示一个屏幕，允许选择要在进程列表中显示的信息列。带星号(\*)的列是选定的显示列，使用上下箭头键在列之间移动，按空格键选择或不选择该列 (星号的有无将显示该状态)。此外，如果按字符 s (位于某一列时)，将使进程列表按该字段排序，该排序将在按 R 键时应用。
- u 或 q 或 U : 选择特定用户的进程。请求用户名；按 Enter 键选择所有用户。
- M : 进程列表按内存使用情况排序。
- P : 进程列表按 CPU 使用情况排序 (默认选项)。
- top 命令也可以从命令行执行，通过传递修改其行为的选项。这种方式用得不多。

```
htop
```

- htop 命令与 top 类似，但更具交互性和功能性。
- 其界面也分为三个部分：头部显示系统信息，进程列表，以及底部显示用于与 htop 交互的功能键信息。

CPU[| 0.7%]

Mem[||||| 221M/996M]

Swp[ 0K/1.69G]

Tasks: 94, 136 thr; 1 running

Load average: 0.09 0.27 0.32

Uptime: 01:20:11

| PID  | USER      | PRI | NI | VIRT  | RES   | SHR   | S | CPU% | MEM% | TIME+   | Command           |
|------|-----------|-----|----|-------|-------|-------|---|------|------|---------|-------------------|
| 2563 | administr | 20  | 0  | 24396 | 3768  | 3132  | R | 0.7  | 0.4  | 0:00.10 | htop              |
| 1967 | administr | 20  | 0  | 427M  | 25172 | 20864 | S | 0.0  | 2.5  | 0:00.35 | xfdesktop --displ |
| 1979 | administr | 20  | 0  | 393M  | 24744 | 19252 | S | 0.0  | 2.4  | 0:00.63 | xfce4-terminal -- |
| 1438 | root      | 20  | 0  | 245M  | 55400 | 27424 | S | 0.0  | 5.4  | 9:36.20 | /usr/bin/X :0 -se |
| 1994 | administr | 20  | 0  | 60548 | 4960  | 4296  | S | 0.0  | 0.5  | 0:00.12 | xscreensaver -no- |
| 1898 | root      | 20  | 0  | 309M  | 20332 | 15188 | S | 0.0  | 2.0  | 0:02.59 | /usr/lib/italc/ic |
| 1963 | administr | 20  | 0  | 158M  | 18368 | 15480 | S | 0.0  | 1.8  | 0:00.21 | xfwm4 --display : |
| 1    | root      | 20  | 0  | 52948 | 6680  | 4812  | S | 0.0  | 0.7  | 0:01.02 | /sbin/init        |
| 443  | root      | 20  | 0  | 38280 | 4804  | 4256  | S | 0.0  | 0.5  | 0:00.19 | /lib/systemd/syst |
| 454  | root      | 20  | 0  | 45040 | 4368  | 2916  | S | 0.0  | 0.4  | 0:00.20 | /lib/systemd/syst |
| 1163 | systemd-t | 20  | 0  | 124M  | 4448  | 3916  | S | 0.0  | 0.4  | 0:00.00 | /lib/systemd/syst |
| 1152 | systemd-t | 20  | 0  | 124M  | 4448  | 3916  | S | 0.0  | 0.4  | 0:00.02 | /lib/systemd/syst |
| 1229 | root      | 20  | 0  | 37080 | 2688  | 2252  | S | 0.0  | 0.3  | 0:00.00 | /sbin/rpcbind -w  |
| 1240 | statd     | 20  | 0  | 37280 | 2840  | 2252  | S | 0.0  | 0.3  | 0:00.00 | /sbin/rpc.statd   |
| 1264 | root      | 20  | 0  | 23356 | 200   | 4     | S | 0.0  | 0.0  | 0:00.00 | /usr/sbin/rpc.idm |
| 1278 | root      | 20  | 0  | 244M  | 3248  | 2440  | S | 0.0  | 0.3  | 0:00.00 | /usr/sbin/rsyslog |
| 1279 | root      | 20  | 0  | 244M  | 3248  | 2440  | S | 0.0  | 0.3  | 0:00.00 | /usr/sbin/rsyslog |

F1Help

F2Setup

F3Search

F4Filter

F5Tree

F6SortBy

F7Nice

-F8Nice

+F9Kill

F10Quit

pstree

- 显示正在执行的过程的层次结构。

- 语法: pstree [选项]

其中一些选项有: -p 显示进程的标识号 (PID) -a 显示命令的参数 -u 显示进程的所有者用户 (括号内)

```
$pstree
systemd--ModemManager--{gdbus}
 --{gmain}
 --NetworkManager--dhclient
 --{NetworkManager}
 --{gdbus}
 --{gmain}
 --acpid
 --agetty
 --anacron
 --applet.py--{gmain}
 --2*[at-spi-bus-laun--dbus-daemon]
 --{dconf worker}]
 --{gdbus}]
 --{gmain}]
```

pstree 示例 (无选项)

pgrep

- 该命令 pgrep 在进程列表中搜索, 根据进程名称查找 PID (类似于 ps | grep)。

- 语法: pgrep [选项] 模式



最常用的选项是 -u，它仅显示属于指定用户的过程。

```
$pgrep -u root cups
1263
1342
$
```

示例 pgrep

pidof

- 命令 pidof 用于查找正在运行程序的进程标识符。例如：pidof getty

kill

- 用于向一个或多个进程发送信号。发送信号者必须是进程的所有者或系统管理员。
- 信号是一个数字代码，尽管它也由一个常量表示。每个信号指示对进程执行的操作（停止它、恢复它等）。

语法：kill [-信号 | -s 信号 | -n 信号] PID

kill [-信号 | -s 信号 | -n 信号] %工作号

kill -l [信号]

- 如果没有指定信号，kill 会向指定的进程发送编号为 15 的信号 SIGTERM、TERM，意图终止其执行。该信号会通知进程自行终止，但进程可以忽略它。如果我们想彻底删除进程，最好发送编号为 9 的信号，该信号无法被忽略。

- 要指定信号，我们可以使用 -signal、-s signal 或 -n signal。
- 要识别要向其发送信号的进程，通常使用 PID（进程编号），但也可以使用 %工作编号（由 bash 分配，与 PID 不同）。

- 带有 -l 或 -L 选项的 kill 命令会列出所有信号。

```
$kill -l
1) SIGHUP 2) SIGINT 3) SIGQUIT 4) SIGILL 5) SIGTRAP
6) SIGABRT 7) SIGBUS 8) SIGFPE 9) SIGKILL 10) SIGUSR1
11) SIGSEGV 12) SIGUSR2 13) SIGPIPE 14) SIGALRM 15) SIGTERM
16) SIGSTKFLT 17) SIGCHLD 18) SIGCONT 19) SIGSTOP 20) SIGTSTP
21) SIGTTIN 22) SIGTTOU 23) SIGURG 24) SIGXCPU 25) SIGXFSZ
26) SIGVTALRM 27) SIGPROF 28) SIGWINCH 29) SIGIO 30) SIGPWR
31) SIGSYS 34) SIGRTMIN 35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```



- 一些最常用的信号包括：SIGHUP / HUP / 1：终端断开连接或控制器进程终止

SIGINT / INT / 2：键盘中断（Ctrl-C） SIGKILL / KILL / 9：立即杀死 SIGTERM / TERM / 15：终止（可控） SIGCONT / CONT / 18：继续。SIGSTOP / STOP / 19：停止（但准备继续） SIGTSTP / TSTP / 20：键盘停止（Ctrl-Z）

- 信号可以通过三种不同方式指定。例如：-9 – SIGKILL -KILL

```
$jobs -l
[1]+ 2494 Parado sleep 500
$kill -SIGCONT 2494
$jobs -l
[1]+ 2494 Ejecutando sleep 500 &
$kill -9 %1
$jobs -l
[1]+ 2494 Terminado (killed) sleep 500
```

kill 示例

- 还存在命令：killall -信号 名称命令

以类似 kill 的方式工作，但不同之处在于不是指定 PID，而是指定程序名称，这将影响所有具有该名称的进程。

```
$killall -STOP sleep

[1]- Detenido sleep 200

[2]+ Detenido sleep 100
$killall -9 sleep
[1]- Terminado (killed) sleep 200
[2]+ Terminado (killed) sleep 100
```

killall 示例

jobs

- 显示当前活动的作业。也就是说，在后台运行且与当前 shell 关联的进程。

- 语法：jobs [选项] [作业 ID]

选项：

- l 显示每个进程的 PID（进程标识符）。-r 显示处于运行状态（running）的进程。
- s 显示处于挂起状态（stopped）的进程。

如果我们传递一个工作编号，它只会显示该工作的信息。

```
$jobs -l
[1]- 2580 Ejecutando mousepad &
[2]+ 2584 Parado sleep 300
$jobs -r
[1]- Ejecutando mousepad &
$jobs -s
[2]+ Detenido sleep 300
$jobs
[1]- Ejecutando mousepad &
[2]+ Detenido sleep 300
```

jobs 命令执行示例

- 命令的输出是：第 1 列：指示分配的工作号（与 PID 不同）。

第 2 列：可以空白，或包含加号(+)，或减号(-)：加号+表示最近停止的作业。

减号-表示倒数第二个停止的作业。

第 3 列：指示作业是否正在运行或已停止(Stopped)。

第 4 列：包含启动进程的命令名称。

bg y fg

- 用于从其作业号恢复一个挂起的任务（作业）。

作业号可以 preceder（前缀）以百分号符号。

- 语法：bg [作业 ID] fg [作业 ID]

° bg 在后台重新开始工作。示例：bg %1 ° fg 在前台重新开始工作。示例：fg 2

没有参数时，作用于最后一个挂起的工作（在命令 jobs 输出的第二列中带有 + 号的那个）

```
$jobs -l
[1]- 2634 Parado sleep 300
[2]+ 2639 Parado mousepad
$bg %1
[1]- sleep 300 &
$jobs -l
[1]- 2634 Ejecutando sleep 300 &
[2]+ 2639 Parado mousepad
$bg 2
[2]+ mousepad &
$jobs -l
[1]- 2634 Ejecutando sleep 300 &
[2]+ 2639 Ejecutando mousepad &
```

bg 示例

```
$jobs
[2]- Detenido mousepad
[3]+ Detenido sleep 400
$fg %3
sleep 400
```

fg 示例

sleep

- 睡眠指定时间。

- 该命令只需等待特定的时间长度。指定的时间可以是整数（秒），也可以是整数后跟字母 s（也是秒）、m（分钟）、h（小时）或 d（天）

- 用于在特定时间段内延迟命令执行。通常用于脚本（shell 程序）。示例：sleep 10 && echo '已经过去了 10 秒'。

wait

- wait [pid...]. 允许在执行子进程时暂停正在后台运行的过程

time

- 语法：time [选项] 命令 [参数]
- 执行指定的命令并返回相对于进程执行的时间，以秒为单位。通常返回三个时间：

real：执行总时间 user：进程执行自身代码所消耗的时间 sys：Unix 系统为该命令所花费的时间。

nice

- 在 Linux 中，进程的优先级由内核自动管理，但允许用户通过 nice 值来修改这些优先级。默认情况下，进程的 nice 值等于零，具有此值时，内核不会修改进程的优先级。
- nice 值是一个介于-20（最高优先级）和 19（最低优先级）之间的数字。只有 root 用户可以给 nice 分配负数并提高优先级。任何用户都可以增加 nice 值并降低进程的优先级。
- nice 命令允许以不同于正常的优先级来执行程序。
- 语法：nice [-n numero\_nice] [命令] [参数]  
          nice [-numero\_nice] [comando] [argumentos]。
- 如果我们在不提供命令的情况下执行 nice，将显示当前的 nice 值。
- 如果我们执行 nice 并提供命令，但没有指定 nice 值，nice 将使用默认值 10。

```
$nice
0
$nice sleep 2000 &
[2] 2912
$nice -5 sleep 500 &
[3] 2922
```

nice 的例子

```
#nice --5 sleep 600 &
[3] 2963
#nice -n -5 sleep 600 &
[4] 2964
#
```

nice 的例子

- 也存在一个命令：renice，它允许在不停止进程的情况下更改进程的优先级。

renice

- 语法：renice [ + | - numero\_nice ] [ -p pid ] [ -u user ] [ -g grp ]

- 其中：

-p pid 更改指定 PID 进程的优先级。

-u user 更改指定用户进程的优先级。

-g grp 更改属于具有 grp ID 的组的用户所执行进程的优先级。

```
$mousepad &
[1] 3200
$renice 10 -p 3200
3200 (process ID) old priority 0, new priority 10
$
```

renice 示例

nohup

- 在 Linux 中，当我们退出登录 shell（注销）或关闭终端时，会向所有子进程发送 SIGHUP 信号。如果我们启动一个后台进程然后退出会话，该进程将在启动它的 shell 死亡时终止。

- nohup 命令允许在后台执行进程，使其忽略 SIGHUP 信号。该命令的输出被重定向到 nohup.out 文件。

- nohup 命令允许在后台执行一个进程（以&结束），以便在关闭会话后继续执行。

- 语法：nohup comando [参数]。例如：nohup sleep 2000 &

```
$nohup sleep 2000 &
[2] 3505
$nohup: se descarta la entrada y se añade la salida a «nohup.out»
$
```

nohup 示例



不错

- 允许更改进程的优先级。默认情况下，所有进程在 CPU 面前的优先级相同，为 0。使用 nice 可以以修改后的优先级启动程序（进程），根据需要设置更高或更低的优先级。
- 优先级范围从-20（最高）到 19（最低）。只有 root 或超级用户可以设置负优先级，这些优先级更高。使用 ps 的-l 选项可以查看显示此值的 NI 列。

```
#> nice (无参数时，显示默认优先级)
```

```
0
```

```
#> nice -n -5 comando (以-5 的优先级启动命令，给予其更多 CPU 时间)
```

renice

- 与 nice 在启动执行时设置进程优先级一样，renice 允许实时修改优先级，无需停止进程。

```
#> nice -n -5 yes (以-5 的优先级运行程序'yes')
```

(dejar ejecutando 'yes' y en otra terminal se analiza con

'ps')

```
#> ps -el
```

```
F S UID PID PPID C PRI NI ADDR SZ WCHAN TTY TIME CMD
```

```
4 S 0 12826 12208 4 75 -5 - 708 write_ pts/2 00:00:00 yes
```

```
#> renice 7 12826
```

```
12826: 旧优先级 -5，新优先级 7
```

```
#> ps -el
```

```
F S UID PID PPID C PRI NI ADDR SZ WCHAN TTY TIME CMD
```

```
4 S 0 12826 12208 4 87 7 - 708 write_ pts/2 00:00:15 yes
```

(请注意第一个情况中的 NI 字段为 -5，第二个情况使用 renice 后变为 7，实时显示)

nohup y &

- 当涉及在后台执行进程时，会使用 nohup 命令或 & 操作符。尽管它们实现的功能相似，但并不相同。
- 如果希望释放终端，以便程序可以运行较长时间，则可以使用 . 这在进程的输出结果不需要发送到标准输入（stdin）时效果最佳，例如执行备份或在控制台或终端中打开 Xwindow 程序时。要实现这一点，只需在命令末尾添加 & 符号即可。

```
$> yes > /dev/null &
```

```
$> tar czf respaldo /documentos/* > /dev/null/ &
```

```
$> konqueror & (使用这些示例，命令将执行并释放终端，返回提示符)
```

- 然而，上述方法会导致被调用的进程的父进程 PPID 成为终端进程本身。因此，如果关闭终端或退出会话，则依赖终端的子进程也会终止，这并不方便，如果希望进程继续执行。
- 为解决这个问题，可以使用 nohup 命令，它允许像 '&' 一样将进程放入后台执行，并使其免受挂起（因此得名 nohup）的影响，即当执行该进程的终端或控制台挂起或终止时。

```
$> nohup yes > /dev/null &
```

```
$> nohup czf respaldo /documentos/* > /dev/null/
```

```
$> nohup konqueror
```

这样就可以避免在关闭控制台时进程被“挂起”。

jobs

- 例如，如果只有访问一个单一的控制台或终端，并且需要执行多个将长时间运行的命令，那么可以像之前看到的 nohup 和 & 运算符一样，将它们发送到后台运行，以释放终端并继续工作。

- 但如果只在终端中，这可能难以控制，为此我们有 jobs 命令，该命令列出当前进程

正在运行：

```
#> yes > /dev/null & [1] 26837 #> ls -laR > archivos.txt & [2] 26854 #>
jobs [1]- Running yes >/dev/null & [2]+ Running ls --color=tty -laR /
>archivos.txt &
```

- 在之前的示例中，执行了 yes 命令并将其发送到后台 (&)，系统返回[1] 26837，表明这是任务[1]及其 PID，第二项任务同样是一个从根目录开始的递归列表，并将其发送到文件，这是第二项任务。
- 使用 fg（前台）和 bg（后台）命令可以操作暂时挂起的进程，无论是由于收到了暂停信号 STOP（20）还是因为在执行时按下了 ctrl-Z。要重新在前台恢复其执行，我们将使用 fg：

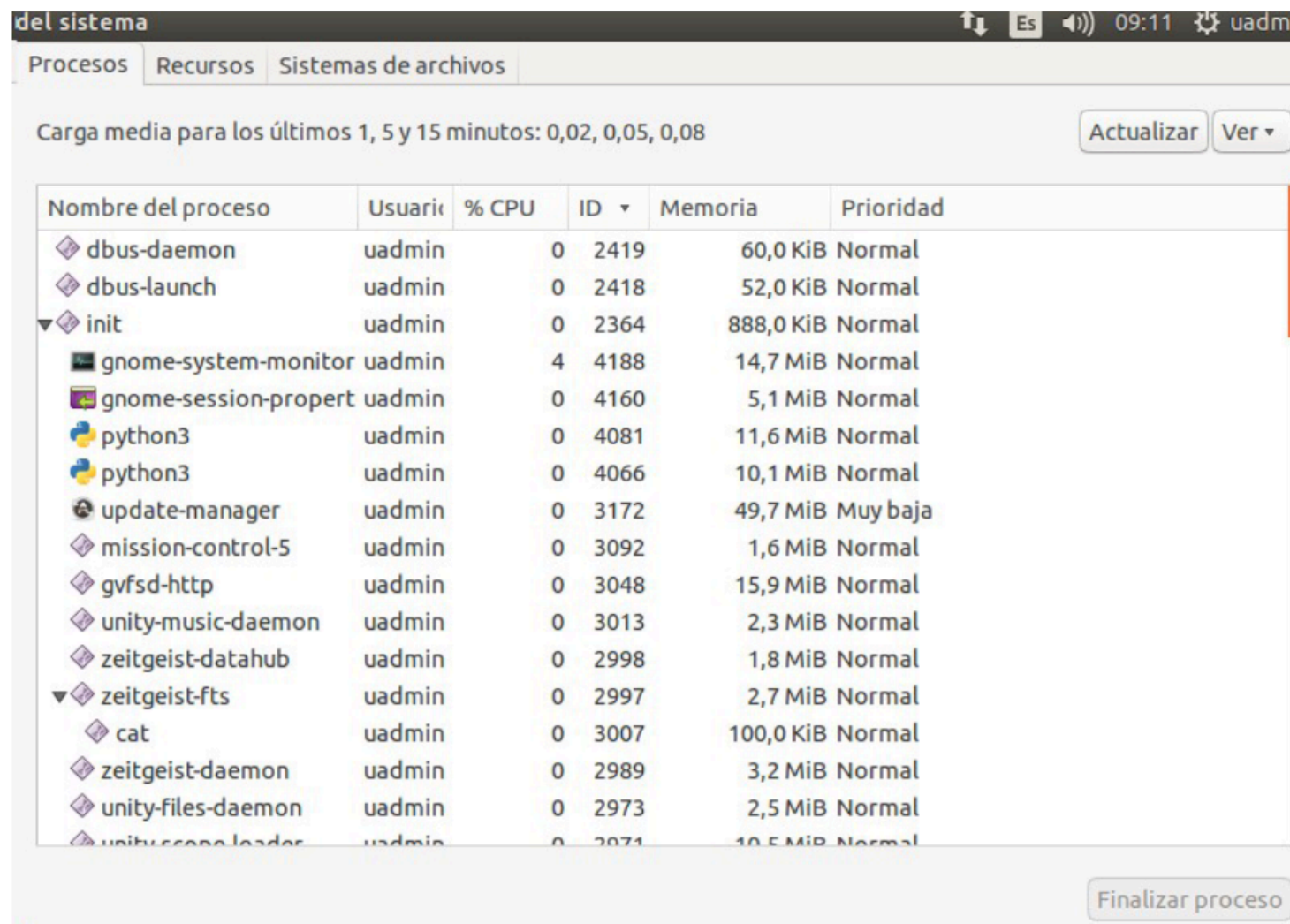
```
#> jobs
[1]- 已停止 yes >/dev/null & [2]+ 已停止 ls --color=tty -laR / >archivos.txt
& #> fg %1 #> jobs [1]+ 正在运行 yes >/dev/null & [2]- 已停止 ls --
color=tty -laR / >archivos.txt &
```

请注意，当将'作业'或进程 1 带到前台时，它获得了[+]符号，表示它现在在前面。使用 bg 也可以做到同样的事情，但会将进程重新启动到后台。此外，也可以使用 kill 命令杀死进程，只需指定 jobs 返回的编号即可：kill %1，这将终止 jobs 编号为 1 的进程。

## 2.2. 图形界面中的进程管理。

- 允许查看/管理进程的图形环境应用程序是系统监视器。它还监视系统状态，显示正在使用的处理器、内存和磁盘空间。

- 系统监视器可以通过 `gnome-system-monitor` 命令从命令行启动。



° 点击“进程”标签页将显示进程列表。

- 在“查看”按钮中，我们可以选择查看所有进程、活动进程或当前用户的进程。查看依赖关系。
- 可以选择显示在每个进程上的信息字段 右键点击标题 → 选择/取消选择字段（进程名称、状态、%CPU、）
- 右键点击一个进程，可以停止进程、继续、结束、杀死...

° 在系统监视器中我也可以看到：

- 在“资源”选项卡中显示一个屏幕，展示 CPU 使用情况、内存使用情况和网络使用情况的信息
- 在“文件系统”选项卡中显示有关文件系统的各种信息。

## 2.3. 虚拟目录 /proc

/proc 是一个虚拟文件系统，存储有关系统和进程的信息。它具有以下特点：

- 在启动时初始化
- 在内存中实现，不存储在磁盘上



- 目录结构 /proc 取决于内核版本
- 进程命令 (ps、top 等) 获取此目录下进程的信息
- 基本概念是，系统中的每个进程都会创建一个目录在 /proc 中。目录的名称是进程的 PID。在这个目录中有包含进程信息的文件。例如，/proc/619 是 PID 为 619 的进程对应的目录。

这些目录中的一些文件包括：

- cmdline 文件，包含启动该进程时的命令行。
- cwd 指向进程当前工作目录的符号链接。
- status 文件，包含进程的状态信息。
- exe 是指向可执行文件名的链接。
- environ 是一个包含进程环境变量的文件。◦ fd 包含已打开文件、文件或相关进程的描述符的目录。
- /proc 中还有其他文件和目录，它们包含有关 CPU、磁盘分区、设备、中断向量、内核计数器、文件系统、加载的模块等广泛信息。其中一些文件和目录包括：
  - cpuinfo：CPU/处理器静态信息
  - meminfo：内存使用信息
  - partitions：分区信息
  - filesystems：内核支持的文件系统
  - version：内核版本和日期
  - bus/：包含 PCI 和 USB 总线信息的目录
  - cmdline：内核启动行
  - devices：系统字符或块设备
  - modules：内核模块
  - net/：包含网络信息的目录
  - interrupts：显示每个 IRQ 的中断次数
  - ioports：列出系统使用的输入输出端口

- 无特权的用户程序可以安全地读取大部分这些信息，以了解系统行为。可以写入这些文件中的任何一个来修改系统参数。

## 2.4. 特殊权限文件

我们已经知道：

- 文件/目录的基本权限是：r, w, x。并且这些权限是针对所有者、所属组和其他用户设置的（一个文件有一个所有者和一个所属组）。
- 进程也有一个所有者和一个所属组。使用普通权限时，它们分别是启动执行的用户的主用户和主组。

文件可以有特殊权限，这些权限会影响其执行方式，从而影响进程。这些权限包括：

suid (SUID 位) chmod u+s 文件→

- 对于可执行文件，会在用户级别改变权限。在执行期间，进程的有效用户是文件的所有者，而不是启动进程的实际用户。
- 也可以使用数字表示法设置 SUID 位，在左侧添加一个四位数字，SUID 位的值是 4 (4000)
- 在使用 ls -l 命令时可以识别：在用户权限中，会出现 “s” 而不是 “x”。如果出现 S，则表示没有执行权限，在这种情况下，权限无效。
- 实用示例：

```
* El ejecutable "gestorbd" lee el fichero "basedatos":
 _rwxr_xr_x root root /opt/bin/gestorbd
 _rw_____ root root /opt/datos/basedatos
* El usuario pilar puede ejecutar "gestorbd" pero NO leer "basedatos"
* Pilar SÍ podrá leer "basedatos" si "gestorbd" tiene los permisos:
 _rwsr_xr_x root root /opt/bin/gestorbd
```

目的：当可执行文件具有 SUID 时，执行该文件时，进程将获取文件所有者的权限（通常是 root），而不是启动该进程的用户权限。

- 典型用法：允许用户执行需要特定权限的任务，而无需给予他们完整的 root 权限。
- 典型示例：/usr/bin/passwd（所有者为 root，具有 SUID 权限）可以修改/etc/shadow。

sgid (SGID 位) chmod g+s → 文件

- 对于可执行文件，更改域级别为组。在执行期间，进程的有效组是文件的组，而不是执行该文件的用户的实际组。

- 实用示例：

```
* El ejecutable "gestorbd" lee el fichero "basedatos":
 _rwxr_xr_x root root /opt/bin/gestorbd
 _rw_rw_____ root root /opt/datos/basedatos
* Los miembros del grupo "alumnos" pueden ejecutar "gestorbd"
 pero NO leer "basedatos"
* Ahora SÍ podrán leer "basedatos" con los permisos:
 _rwxr_sr_x root root /opt/bin/gestorbd
```

- 对于目录，在其内部创建文件时，新文件的组所有权是目录的组，而不是执行命令的用户的组。

```
* Si el usuario "pilar" sólo pertenece al grupo "profesor"
* Se tiene el directorio "drwxrwsr_x pilar alumnos /practicas"
* Si "pilar" ejecuta "cp tema2.pdf /practicas" entonces el
 fichero copiado pertenecerá al grupo "alumnos":
 rw_r__r__ pilar alumnos /practicas/tema2.pdf
```

- 实用示例：

- 还可以使用数字表示法设置 SGID 位，在左侧添加一个四位数字，SGID 位的值是 2（2000）
- 使用 ls -l 命令时可以发现：组权限中显示为“s”而不是“x”。

可执行文件中的目的：进程执行时会继承文件的 GID，这对于共享组资源很有用。

目录中的目的：在该目录内创建的新文件/目录会继承目录的 GID，便于在共享项目中协作。

典型用法：

- 可执行文件：需要访问特定组资源的工具。
- 共享工作目录：保持所有文件属于同一组。

项目目录：使用 chmod g+s /proyectos/equipo，以便创建的所有内容都属于该目录的组。

t (粘滞位) chmod +t 文件→

- 对于可执行文件 保持文件映像在内存中→执行完毕后。如今不再用于此功能
- 对于目录 如果你有写入目录的权限，→你可以创建文件但只能删除属于你的文件
  - 目录/tmp 的权限为 drwxrwxrwt
  - 只有在它对所有用户都有写入权限（drwxrwxrwx）时才有意义
- 也可以使用数字表示法设置粘滞位，在左侧添加一个四位数字 1xxx→
- 使用 ls -l 命令时可以识别：出现 “t”  
目的（在目录中）：只有文件的所有者（或 root）可以删除或重命名其目录中的文件，即使其他人具有该目录的写入权限。
  - 典型用途：公共和共享目录，其中许多用户进行写入操作

更多参考资料：

<https://es.wikipedia.org/wiki/Setuid>

[https://es.wikipedia.org/wiki/Sticky\\_bit](https://es.wikipedia.org/wiki/Sticky_bit)

摘要

- SUID：以所有者的 UID 执行（拥有者的权限）。  
安全性：如果使用不当，风险很高。
- SGID：以文件所属 GID 执行；在目录中，为新建文件继承 GID。非常适合协作。
- Sticky：保护目录中的文件；只有文件所有者才能删除。



## 2.5. 其他命令 - 系统性能

除了 ps 和 top 之外，还有一些基本命令可以显示系统在 CPU 使用和内存消耗方面的状态：

- uptime：显示当前时间、系统已运行时间、当前连接用户数以及过去 1 分钟、5 分钟和 15 分钟的平均系统负载（与 top 的第一行相同）。示例：

```
$ uptime
11:28:43 up 13 min, 5 users, load average: 0,00, 0,03, 0,05
```

- w：除了 uptime 提供的信息外，w 命令还显示有关用户及其进程的信息

示例：

```
$ w
11:29:24 up 14 min, 5 users, load average: 0,00, 0,02, 0,04
USER TTY FROM LOGIN@ IDLE JCPU PCPU WHAT
pepe tty1 11:19 9:16 0.08s 0.05s -bash
administ :0 :0 11:17 ?xdm? 6.93s 0.00s /bin/sh /etc/xd
administ pts/0 :0 11:18 11:22 0.03s 0.03s bash
administ pts/1 :0 11:18 0.00s 0.03s 0.00s w
alumno tty2 11:20 9:06 0.08s 0.04s -bash
```

定义：

° LOGIN 用户连接的时间 ° IDLE 终端空闲的时间 ° JCPU TTY 上运行的过程所消耗的 CPU 时间 ° PCPU 当前进程消耗的时间（出现在 WHAT 列中的进程）

- free：显示系统中空闲和使用的内存量，包括物理内存和交换内存，以及内核使用的缓冲区

（类似于 top 头部显示的内容）

示例：

```
$ free
 total used free shared buffers cached
Mem: 1020372 577120 443252 8648 43064 265292
-/+ buffers/cache: 268764 751608
Swap: 1768444 0 1768444
```

° 共享列没有任何意义（已过时） ° 选项：

- -b,-k,-m,-g 内存以字节/K 字节/M 字节/G 字节为单位
  - -t muestra una línea con el total de memoria (física + swap)
  - -s delay muestra la memoria de forma continua, cada delay segundos
- Se puede instalar el paquete sysstat que contiene una colección de herramientas de monitorización de rendimiento. Pueden proporcionar datos instantáneos de rendimiento o almacenarlos como históricos para futuras consultas. Especialmente en entornos de servidor, sus datos proporcionan información muy valiosa sobre posibles cuellos de botella de nuestro sistema. Alguna de las herramientas que incluye son:
    - ° iostat: Estadísticas de CPU, de dispositivos de E/S, de particiones y del sistema de archivos
    - ° vmstat: Informa sobre uso de la memoria física y virtual,
    - ° mpstat: 系统每个处理器的性能统计
    - ° pidstat: Linux 进程统计
    - ° sar: 收集、报告和保存系统活动信息 (CPU、内存、磁盘、中断、网络接口、TTY、内核表等)

-----

last / lastb / lastlog → 显示最后访问/访问失败/用户最后一次连接  
记住 df、du 命令（去年见过），用于检查磁盘性能

### 3. LINUX. 启动过程 – 服务管理

每当修改计算机的启动方式时，都存在无法获得预期结果的可能性。在这种情况下，了解如何查找更多关于启动过程中发生的事情的信息会很有用。

#### 3.1. 启动过程 - 阶段

Linux 的启动过程是操作系统在用户打开计算机时启动的过程。它负责系统的初始化和设备的管理。包括从计算机启动到用户界面完全可操作的所有内容。

了解启动过程的步骤将有助于解决问题并调整计算机的性能。

一个 x86 系统启动过程的基本步骤是：

### 1. 硬件启动：

系统启动，此时一个特殊的硬件电路使 CPU 读取特定地址并执行该地址存储的代码。该地址存储着固件（BIOS 或 EFI），因此 CPU 会执行它。

固件执行一些任务，包括硬件检查（POST）、配置以及寻找引导加载程序。

### 2. 引导加载程序（Boot Loader）。控制权从固件转移到引导加载程序。

引导加载程序通常存储在系统的某个硬盘驱动器中，或是在引导扇区（用于传统 BIOS/MBR 系统）或 ESP 分区（用于 EFI-UEFI 系统）。

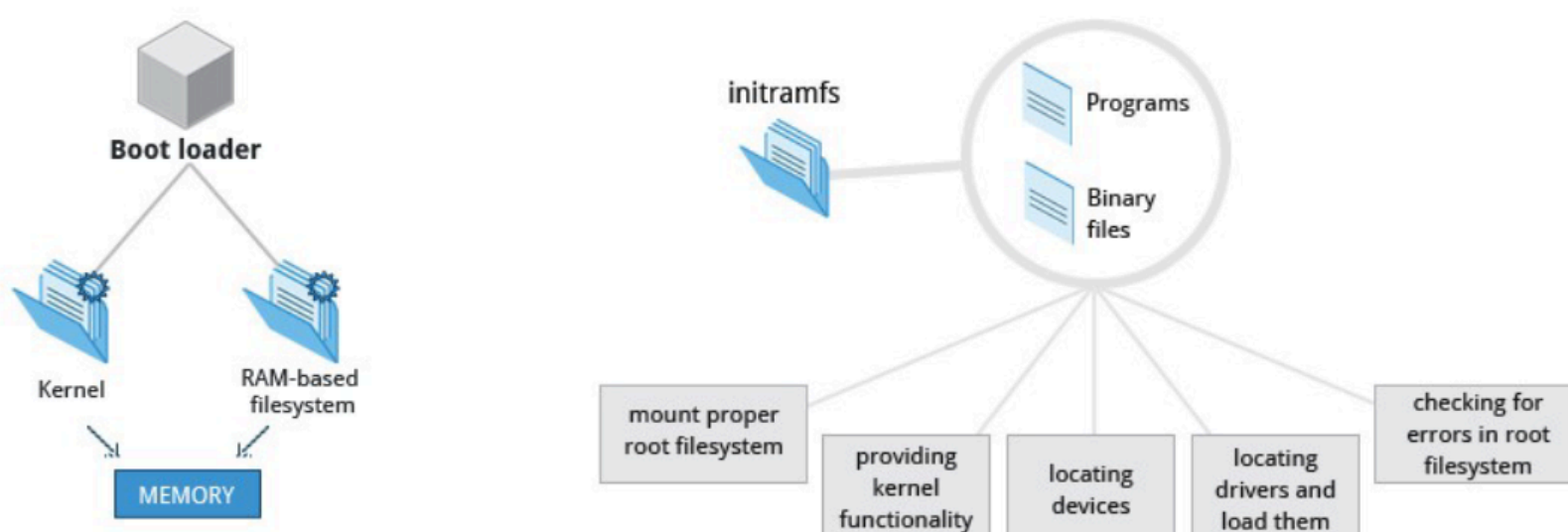
在 Linux 中，最常用的引导管理器是 GRUB。它会向用户显示一个界面，用于选择要启动的操作系统或内核。选择操作系统后，引导管理器会加载所选内核，执行命令 `/boot/vmlinuz-`，并加载一个初始磁盘/文件系统-RAM（`initramfs`）到内存中。

### 3. Linux 内核 + `initramfs`。

当 Linux 内核接管控制权时，它会初始化和配置计算机内存，并配置硬件（处理器、存储设备等）。它还会加载一些必要的用户应用程序。

`initramfs` 包含执行多种任务（其目的是挂载根分区）的程序和二进制文件。它负责通过 `udev` 工具加载未在内核中编译的存储设备驱动程序。它仅会加载访问根分区所需的模块；其余模块将在 `init` 阶段稍后加载。

程序 `mount` 会通知操作系统哪些文件系统已经准备好使用，并将它们与一个挂载点关联。如果正确，`initramfs` 会从 RAM 中删除，并加载和执行系统初始化程序，默认为 `/sbin/init`。



#### 4. 初始化进程 /sbin/init 和服务。

程序 /sbin/init（也称为 init）接收进程 ID（PID）1，因为它是在系统中运行的第一个程序。它将启动其他进程以确保系统运行（将是系统所有进程的来源，除了内核进程）。

Linux 系统包含许多在后台运行的程序以向系统提供服务；在 Linux 术语中，这些被称为守护进程（daemons）。程序 init 负责在启动 Linux 时启动所有这些程序。这是所谓的初始化进程。

Linux 的原始 init 程序基于 System V（SysV），它使用一系列分为多个执行级别的脚本来确定每个时刻运行哪个程序。

SysV 的 init 程序使用了多年，但随着系统复杂性的增加和需要新服务，执行级别的脚本变得复杂，这导致寻求替代方案。

目前大多数 Linux 系统已经用 systemd 初始化进程替换了 SystemV init。第一个运行的程序，PID 为 1，是 systemd。

它的主要目标是控制 Linux 中由热插拔设备动态生成的环境。

#### 5. 文本模式登录。

在启动过程接近结束时，init 会启动一些文本模式登录指示器（由一个名为 getty 的程序完成）。这些指示器允许输入用户名和密码，并启动一个 shell（通常是 bash），等待输入命令。



大多数发行版提供六个文本终端，通过 ALT + F1、ALT+F2 等访问，以及一个图形终端。（通过 CTRL + ALT + F1 访问终端）

## 6.X 窗口系统

通常，在 Linux 桌面系统中，X 窗口系统作为启动过程的最后一步被加载。一个名为屏幕管理器（display manager）的服务会加载 X 服务器（之所以这样称呼是因为它为应用程序，即 X 客户端提供图形服务）。它还负责图形登录并执行用户登录后的适当桌面环境。

如果屏幕管理器没有默认启动，在文本模式控制台登录后，我们执行 startx 命令。

## 3.2. 提取启动过程信息

存在一些内核和模块日志信息（为了使内核体积不至于过大，大多数选项都编译为模块，在需要时再加载），这些信息存储在内核的循环缓冲区中。默认情况下，Linux 在启动过程中会显示发送到循环缓冲区的消息；这些消息在屏幕上滚动速度太快，以至于无法阅读。我们可以使用 dmesg 命令来检查这些信息：选项 -H 将其以人类可读的格式显示（还有更多选项...）。

另一种已记录的信息来源是由系统日志守护进程 rsyslogd（它是 syslogd 的改进版本）生成的。日志文件包含操作系统（内核、运行的服务和应用程序）的活动消息。这些日志文件位于 /var/log 子目录下，而 rsyslog 的日志通常位于 /var/log/syslog：默认情况下，它包含最多的操作系统信息。/var/log 中的其他日志文件也可能包含有用信息，因为某些服务会直接将日志信息写入其日志文件。

启动缓冲区循环消息或 /var/log 文件中的消息可能会产生大量输出，因此建议通过 less 分页器进行管道处理或重定向到一个文件：dmesg | less 或 dmesg > boot.messages

注意 !!!

如何使用 Journalctl 查看和操作 Systemd 的日志 | DigitalOcean

### 3.3. System V 初始化过程

需要配置初始化过程，以便根据希望在系统中执行的具体功能来启动程序。例如，Linux 服务器并不总是需要启动图形桌面环境，或者 Linux 桌面环境也并不总是需要启动 Apache Web 服务器。

SystemV 初始化过程，也称为 SysV，是 Linux 的传统方式（继承自 UNIX 的 System V）。SysV 初始化过程的关键是执行模式/级别。程序 init 根据系统的执行级别决定启动哪些服务/守护进程。

执行级别从 0 到 6 编号，每个级别都分配了一组应该处于活动状态的服务/守护进程。启动时，Linux 会进入默认的执行级别，该级别可以配置。

-

下表总结了执行级别的常规用途：

| 执行级别<br>执行 | 目的                                                                                                           |
|------------|--------------------------------------------------------------------------------------------------------------|
| 0          | 执行级别 0 设置为关闭/停止系统。在具有现代硬件的设备上，系统会完全关闭。                                                                       |
| 1,单用户模式。   | 启动守护进程/服务<br>执行维护任务（例如调整分区大小）所需的最小配置。启用一个用于 root 的终端。                                                        |
| 2 a 5 级，   | 根据每个 Linux 发行版的需求进行配置。<br><br>通常情况下，它们都是多用户系统。在某些发行版中，级别 2 被配置为多用户且无网络服务，级别 3 为多用户且有网络服务，而级别 5 用于启动带图形界面的系统。 |
| 6          | 此级别用于重启系统。系统完全关闭，计算机将自动重启。                                                                                   |

每个执行级别都关联一个 `/etc/rc?` 的目录（其中?是执行级别的编号）。在这些目录中，我们可以找到指向控制守护进程/服务的脚本的符号链接，这些脚本位于 `/etc/init.d/`。

例如，我们看到与该级别关联的目录内容

```
$ls /etc/rc1.d/
K01alsa-utils K01lwsmd K01winbind S01bootlogs
K01atd K01network-manager K02avahi-daemon S01killprocs
K01cups-browsed K01nmbd K02cups S01motd
K01exim4 K01samba-ad-dc K04rsyslog S02single
K01hddtemp K01saned K06nfs-common
K01irqbalance K01smbd K06rpcbind
K01lightdm K01speech-dispatcher README
```

执行 1：

这些符号链接的第一个字母表示要执行的操作：

- K：如果链接以 K（kill）开头，表示停止守护进程。
- S：如果链接以 S（start）开头，表示启动守护进程。

在这段文字之后有一个两位数，介于 00 和 99 之间，它表示脚本执行的顺序。这个顺序很重要，因为某些守护进程需要在其他守护进程运行后才能启动。最后出现的是要启动或停止的相关守护进程的名称。

## SystemV - 服务管理

因此，尽管守护进程/服务与其他在后台运行的过程一样，但其管理方式和调用方式与其他系统命令和程序不同。通常，守护进程在目录 `/etc/init.d` 中有一个脚本（也称为 shell 脚本或 shell 脚本），允许通过以下语法启动它们、停止它们或查看其执行状态：

`/etc/init.d/服务名称 操作`

守护进程脚本应接受的基本操作参数是：

- 启动：启动守护进程
- 停止：停止守护进程
- 重启：重启守护进程，用于使其重新读取配置文件。
- 重新加载：虽然不是所有守护进程都支持，但此操作允许在不停止守护进程的情况下重新加载配置文件。
- 状态：显示当前状态

例如，要启动 Samba 服务器，我们将执行：

`/etc/init.d/smb start`

目录 `/etc/default` 包含定义守护进程配置变量的文件。

许多发行版也提供了 `service` 命令来启动/停止服务。语法：

`service nombre_del_servicio 动作`

示例：# `service atd status`

`update-rc.d`

- 修改执行级别的配置可以手动进行：将服务处理脚本包含在 `/etc/init.d` 目录中，并在每个执行级别的目录中创建符号链接。在 Debian 及其衍生系统中，也可以通过 `update-rc.d` 命令更方便地完成。



自动创建每个执行级别所需的符号链接（在其他发行版中，命令可能不同）。

- 命令 `update-rc.d` 用于启用/禁用特定运行级别下的服务

- 语法: `update-rc.d [-n] [-f] remove`

`update-rc.d [-n] <basename> defaults`

`update-rc.d [-n] <basename> disable | enable [S|2|3|4|5]`

选项 `-f` 表示强制删除，选项 `-n` 则什么也不做，只显示将要执行的操作。

`runlevel`

- 该命令告诉我们之前的执行级别和当前的执行级别。
- 执行该命令会返回两个字符：第一个字符是之前的执行模式。当字符为 `N` 时，表示系统自启动以来没有改变执行模式。第二个字符指示当前的执行级别。

`init`

- `init` 命令允许改变系统的运行级别。
- 通过传递要切换到的运行级别作为参数来执行。例如：`init 6`

`shutdown`

- 此命令用于停止、关闭或重启系统。它是执行这些操作最合适的命令。
- `shutdown` 命令向所有用户发送标准消息。它还允许指定系统何时关闭，以便用户可以结束他们的进程。
- 语法: `shutdown [选项] [时间] [消息]`  
最常用的选项是: `-r` 重启系统。

`-P` 关闭系统（这是默认选项） `-H` 停止系统。

`-c` 取消待处理的关机。

`-k` 不停止、不关闭或不重启；仅显示消息

关闭或重启；只需输入消息即可 • 最简单的方式是传递临时参数 `now`。我们会执行 `shutdown now`。或者，我们可以指定向所有已连接用户发送的消息。

- 时间参数也可以用 `hh:mm` 格式指定，用于 24 小时制时间格式，以及 `+m`（或不加 `+` 符号）来指定从当前时间起增加 `m` 分钟的时间段。

示例：`shutdown -r +10` “系统将在 10 分钟后重启”

- 还存在 `halt`、`reboot` 和 `poweroff` 等命令。
  - `halt` 相当于：`shutdown -h now`
  - `reboot` 相当于：`shutdown -r now`

### 3.4. 初始化进程 `systemd`

`Systemd` 被创建以提供比 `SysV` 更快、更灵活的启动。

`Systemd` 的初始化方法在 Linux 世界中越来越被使用。目前它是 Fedora、CentOs、Debian、Ubuntu 等发行版中使用的默认初始化进程。

`Systemd` 的优点：

- 允许并行启动进程，使用套接字并减少启动时间
- 允许按需激活服务或与事件关联的服务，通过 D-Bus。
- 管理服务之间的依赖关系。
- 监控和记录进程到自己的日志中，可过滤。
- 优化资源使用，利用 `cgroups`
- 更直观的配置语法

兼容 `SystemV` 脚本，但它也有反对者，并收到了来自社区成员的多次批评，他们认为 `systemd` 与 Linux 的基本哲学相违背：“做简单的事，并把它做好”。

在 `systemd` 系统中，`init` 程序被 `systemd` 取代，其 PID 为 1。它们支持 `SysV` 的惯例以提供兼容性，但不使用 `/etc/inittab` 文件。

`systemd` 中最基本的概念是 Unit（单元）：

- 一个 Unit 代表一个 `systemd` 可管理的资源。

使用 systemd 可以管理系统的许多配置方面：服务、挂载点、日志、系统状态等。

° Unit 的定义通过 Unit 文件实现：

- 它们允许定义和管理每个 Unit 的方面。
- 包含与 Unit 类型相关的指令。

° 每种 Unit 类型都由其 Unit 文件的后缀标识。

例如，在服务管理中，使用后缀 .service；在设备管理中，使用后缀 .device。

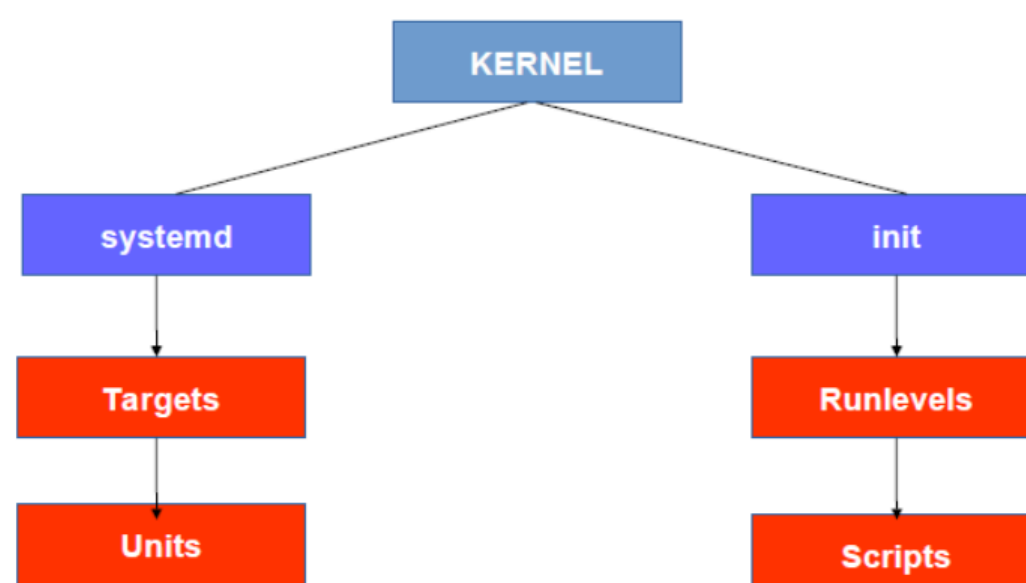
° 通过 units 管理的一个重要方面是系统服务和守护进程。

systemd 的初始化过程使用 target 类型的 units，与 SysV 使用执行级别的方式类似，但作用略有不同。每个 target 都是以命名而非编号的方式指定，并设计用于执行特定目的。每个 target 代表系统需要运行的一组不同的服务。与 SysV 的执行级别相比，另一个区别是 systemd 中的系统可以同时激活多个 target。target 之间存在依赖关系，因此它们相互关联。

default.target unit 等同于 SysV 中的默认执行级别概念。这是 systemd 启动时查找的 unit。通常设置为指向 /lib/systemd/system 目录中某个 target 文件的链接。

为了便于从 SysV 过渡到 systemd，存在一些目标 (targets)，它们模拟了 SysV 的标准执行级别 0-6，名称介于 runlevel0.target 和 runlevel6.target 之间。

## Systemd vs SysV



### 3.4.1. 单元文件管理

Unit 文件具有 `nome_unit.tipo_unit` 类型的名称，例如 `ssh.service` 或 `multi-user.target`。

Unit 类型：° `service`：服务管理。

° `socket`：管理其他 Unit 使用的套接字（如服务）。

° `device`：设备管理（使用 `udev`）° `mount` e `automount`：挂载点管理。

° `swap`：交换空间管理。

° `target`：目标管理。 . 允许将多个 units 分组。  
可以同时启动它们。

° `path`：用于基于所引用的文件系统路径的事件激活 `units.service`。

° `timer`：任务调度（类似于 `cron` 或 `at`）° `snapshot`：系统快照管理。

° `slice`：使用 `cgroup` 管理进程相关资源。

° 范围：管理由 `systemd` 组织和创建的进程。

单元文件的定义语法基于章节。每个章节指明一个方面，通过指令定义与该方面相关的所有参数。单元文件的一些章节包括：

° `[Unit]`: 定义单元信息及其与其他单元的相互关系。

- 指令： `After` / `Before` 在列出的单元之后/之前执行； `→` `Wants` / `Require` 建立较少/严格的依赖关

° `[安装]`: 定义管理 units 的指令，建立关系，例如 unit 与...的关系

目标 会员。

° `[服务]`: 定义服务的特定指令。

° `[Socket]`: 定义与服务相关联的套接字配置指令。

° `[Mount]` e `[Automount]`: 定义挂载点的指令。

° `[Swap]`: 定义并启用用于匿名虚拟内存页面的交换空间的指令。

° `[Timer]`: 定义和管理时间事件。



- ° [路径]: 文件系统监控。
- ° [切片]: 管理资源分配给进程。

systemd 系统将单元的配置文件存储在 `/lib/systemd/system` 或 `/etc/systemd/system` 目录中。

\*.wants 目录（位于这两个目录中）包含指向特定服务的符号链接。这些服务将与对应目录 wants 的目标一起运行。记住，如果一个目标需要另一个目标，那么另一个目标的服务也会被加载。

让我们看一个单元文件示例，这次是 `ssh.service`：

```
#cat
/lib/systemd/system/ssh.service

[Unit] Description=OpenBSD Secure
Shell server After=network.target
auditd.service
[Service]
EnvironmentFile=/etc/default/ssh
ExecStart=/usr/sbin/sshd -D
$SSHDT_OPTS ExecReload=/bin/kill -
HUP $MAINPID KillMode=process
Restart=on-failure
```

```
[Install] WantedBy=multi-
user.target Alias=sshd.service #
```

此文件定义了要执行的程序（`/usr/sbin/sshd`），以及其他选项，例如在启动 `sshd` 服务之前要执行的服务（`After` 行），系统应在哪个目标级别运行（`WantedBy` 行），以及如何重新加载程序（`Restart` 行）。目标单元（`target units`）也使用配置文件，这些文件不定义程序，而是定义要启动的服务单元。

如我们所见，配置文件由方括号内的名称标识的章节组成，后面跟着将值分配给变量的赋值。

### 3.4.2. systemd 服务管理 -> systemctl

在使用 systemd 初始化进程时，会使用程序 systemctl 来管理 Units。我们可以控制服务和其他目标等。该程序接受多种选项和命令。通常需要传递一个 Unit 的名称，即要操作的服务名称（或其他资源）。如果没有指定后缀，systemctl 会假定它是 .service。

systemctl 的语法：

systemctl [选项] 命令 [unit]

systemctl 中与服务相关的最重要的命令有：

- ~~list 显示所有已激活单元的当前状态。~~如果没有指定任何参数，则默认使用此命令。
- start nombre 启动指定单元。→
- stop nombre 停止指定单元。→
- restart nombre → 使指定单元关闭并重启。
- reload nombre 使指定单元重新加载其配置文件。
- status nombre 显示指定单元的状态（可以传递 PID 而不是名称）。
- enable nombre 设置单元以便在计算机下次启动时启动。
- 禁用 nombre 配置单元，使其在下次计算机启动时不启动。
- is-enabled nombre 显示服务是否配置为在启动时自动启动。
- list-unit-files 显示所有可用的单元。

示例 停止服务→

sshd:

```
systemctl stop ssh.service
```

systemctl 相关的最重要命令，与 targets 相关的有：

- isolate name 启动 target unit 并停止所有其他 unit→
- 默认 将系统默认目标单元更改为 target
- get-default 显示系统默认目标单元。
- Set-default name 设置系统默认目标单元。

示例 要切换正在运行的单元，需要使用→ isolate 命令。因此，要访问单用户模式：

```
systemctl isolate rescue.target
```

要返回到系统的默认目标，请使用命令 default。

在 systemd 中，要重启、关闭系统，我们可以使用：

- systemctl poweroff 关闭系统→
- systemctl reboot 重新启动系统→
- systemctl suspend 暂停系统→

### 3.4.3. systemd journalctl→

systemd 初始化过程最具争议的特点之一是它不使用 Linux 标准的 rsyslogd 日志系统，而是拥有自己的日志文件，这些文件不以文本格式存储。journald 守护进程收集所有可用来源的数据，并以二进制格式存储，以便轻松和动态地操作。

要查看 systemd 的日志文件，需要使用 journalctl 命令。

- journalctl -b 显示自上次重启以来的日记条目
- journalctl -u unit 显示由该单元生成的日记条目
- journalctl -k 仅显示内核消息→

## 4. LINUX。任务自动化

在本节中，我们将看到允许自动化重复性任务的命令的使用：

- at、batch 允许在特定时间或特定条件下执行任务
- cron 允许按定期间隔运行工作；安排 cron 任务

### 4.1. 在特定时间执行任务：at

- at 命令允许指定想要执行任务的时刻
- 语法：at [选项] TIME

- 执行 at 命令后，我们切换到新的提示符，这使我们能够输入将在指定时间执行的命令：
  - 保存工作并退出 CTRL-D • 完成后，标准输出将作为邮件发送给用户（如果已安装邮件代理） • 工作将在系统在指定时间开启时执行。

- 示例：

```
$ at 11:45
warning: commands will be executed using /bin/sh
at> ls /tmp > lista
at> env DISPLAY=:0 zenity -info -text="Hola"
at> <EOT>
job 4 at Wed Nov 16 11:45:00 2005
```

- 一些选项：
  - -f FILE 指定一个包含要执行的操作的文件，而不是标准输入
  - -m 在作业执行后向用户发送邮件，即使没有输出。
  - -M 从不向用户发送邮件。
- TIME 可以以多种方式指定：
  - HH:MM 例如 12:54
  - 上午/下午 HH:MM，例如 1:35 下午
  - HH:MM MMDDYY，例如 1:35PM 122505
  - 现在 + 数量单位，其中单位可以是分钟、小时、天，或周：
    - \$ 在当前时间后 2 小时
  - 今天，明天，例如 12:44 明天
  - 午夜（00:00），正午（12:00），下午茶时间（16:00）
- 相关命令：
  - atq o at -l 列出用户待处理的作业。如果是超级用户，则列出所有用户的作业
  - atrm o at -d o at -r 删除通过作业编号标识的作业。
  - batch 在系统负载较低时执行作业：
    - 作业一旦系统负载低于 1.5 即开始



- 负载从文件 `/proc/loadavg` 中获取
  - 负责执行命令的 `at` 守护进程 →
- 配置文件：

管理员可以通过 `/etc/at.allow` 和 `/etc/at.deny` 文件控制 `at` 的使用。列出可以和不可以使用 `at` 的用户。

  - 首先检查 `/etc/at.allow`。如果用户在列表中，则可以使用 `at`。  
如果不存在或文件不存在，则检查 `/etc/at.deny`。如果用户不在 `at.deny` 中，则可以使用 `at`。
  - 如果两个文件都不存在，只有 `root` 可以执行 `at`。◦ 为所有用户授权创建仅空的 `at.deny` 文件

## 4.2. 定期执行任务：cron。

- 要创建定期执行的工作，使用 `cron` 守护进程。
- Cron 响应时间事件。具体来说，“醒来”一次每分钟，检查目录 `/var/spool/cron` 和 `/etc/cron.d` 以及 `/etc/crontab` 的配置文件，如果时间与这些文件中指定的时间一致，则执行这些配置文件中指定的命令。
- Cron 任务有两种类型：系统任务和用户任务。系统 Cron 任务以 `root` 身份运行，并执行整个系统的维护任务。
- Cron 适用于 24/7 运行的系统。如果系统在 Cron 任务执行时关闭，则该次迭代不会执行。问题在于台式机和/或家用系统。补充解决方案：Anacron

### 4.2.1. 用户 Cron 任务

- 要创建用户级 cron 任务，使用 `crontab` 命令（不要与配置文件 `/etc/crontab` 混淆）：
- cron 的使用通过 `/etc/cron.allow` 和 `/etc/cron.deny` 文件进行管理：
  - 如果这些文件不存在，行为取决于系统配置。在 Debian 系统中，如果不存在，所有用户都可以使用 `crontab`
- 任务在 `crontab` 文件中指定，该文件保存在 `/var/spool/cron/crontabs` 中，格式如下：

分钟 小时 日期\_月份 月份 日期\_星期 命令

- 分钟值→ 在 0 到 59 之间。
- 小时值在 0 到 23 之间→
- 日期值在 1 到 31 之间→
- 月份值在 1 到 12 之间→
- 星期值在 0 到 7 之间 (0 或 7 代表星期日)。

此外还可以使用以下字符：

- 星号(\*)表示任何值→
- 逗号(,)用于在一个字段中表示多个值/值列表→
- 连字符(-)用于表示一个范围→
- 斜杠(/)用于表示每隔多少/重复次数→
- 例如：

- 1-5 表示从星期一到星期五在星期值
- 15,45 表示 “一刻钟” 和 “差一刻钟” 分钟
- \*/2 在小时字段中，表示每两小时执行一次 (0、2、4、等)

(此文件可能还包含注释行和变量定义行)

◦ 示例：

- 清除 /tmp 每个工作日的早上 4:30： `30 4 * * 1-5 rm -rf /tmp/*`
- 每小时记录时间，夜间： `*/15 0-8,20-23 * * * echo Hora:$(date)>/tmp/horas`

◦ 也可以使用特殊单词：@hourly, @daily, @monthly, @yearly, @reboot

crontab 命令：创建和编辑周期性任务

• 语法：

- `crontab [-u 用户] [-l|-e|-r]`
- `crontab [-u 用户] 文件` (从文件中安装新的 crontab)

• 选项：

- `-u usuario` 创建或管理特定用户的 crontab (仅 root 用户)
- `-e` 编辑 crontab 文件

- -l                显示 crontab 文件
  - -r                删除 crontab 文件
- 例如，用户将执行命令 `crontab -e` 来编辑其 crontab 文件并创建其任务

#### 4.2.2.      系统定时任务

- 文件 `/etc/crontab` 控制系统定时任务。该文件的行格式如下：

分钟 小时 日期\_月份 月份 星期几 周期次数 命令

- 周期次数是程序执行时使用的账户名称
- 恶魔 cron 会搜索 `/var/spool/cron` 中的文件，并在指定时间执行它们：
    - 此外，它还会执行 `/etc/crontab` 文件和 `/etc/cron.d/` 目录中指定的操作：
  - 这些文件通常用于系统维护
  - 管理员可以创建按小时、每天、每周和每月周期执行的脚本：
    - 只需将这些脚本放置在 `/etc/cron.hourly`、`/etc/cron.daily`、`/etc/cron.weekly` 或 `/etc/cron.monthly` 目录下。
    - 这些脚本的执行日期和时间由 `/etc/crontab` 文件控制。

#### 4.3.      Anacron

- 异步执行计划周期性任务：
  - 系统启动时检查是否有待执行的周期性任务（由于系统关机而未执行）。如果是，则等待一段时间后执行。
- 文件 `/etc/anacrontab` 控制系统的 anacron 任务。该文件的行具有以下格式：

延迟周期 工作 ID 命令

  - 该周期以天数为单位指定（或如 `@monthly`），以及延迟在分钟：

|          |    |              |           |                   |
|----------|----|--------------|-----------|-------------------|
| 1        | 5  | cron.daily   | run-parts | /etc/cron.daily   |
| 7        | 10 | cron.weekly  | run-parts | /etc/cron.weekly  |
| @monthly | 15 | cron.monthly | run-parts | /etc/cron.monthly |

° 在此示例中，每天，cron.daily 中的脚本会在 anacron 启动后 5 分钟执行，每 7 天（等待 10 分钟）会执行 cron.weekly 中的脚本，每月（等待 15 分钟）会执行 cron.monthly 中的脚本。

° anacron 的局限性：

- 仅限管理员使用
- 仅允许按天设置周期（不适用于需一天多次执行的任务）